Attack Simulation – Virtual Lab Development Guidelines

Local-first, isolated, Docker-based cybersecurity simulation architecture

Purpose: provide a practical design guideline for converting an assigned cybersecurity experiment into an interactive virtual lab that can run primarily on a student's own machine. The uploaded college guideline requires interactive simulation/implementation, theory, procedure, assessment/quiz, references and feedback; this document turns those requirements into an implementation workflow.

1. Core Architecture

Use the student's machine as the execution host. The cloud should preferably provide the website, lab package, documentation and (if required) progress data rather than a dedicated compute environment for every student.

Recommended flow:

```
Website / Git repository → Download lab package → Electron Lab Manager → Docker image/container(s) →
Isolated lab network → Target/Database/Simulator → Logs + Metrics → UI + Quiz
```

2. Lab Package Structure

Each experiment should be self-contained so the main application does not need attack-specific implementation details.

```
experiment/ ■■■ lab.yaml ■■■ README.md ■■■ docker-compose.yml ■■■ target/ ■■■ attacker/ ■■■ database/
■■■ scripts/ ■■■ assets/ ■■■ tests/
```

3. Lab Manifest

Use a manifest to tell the Electron application what environment to create. Example:

```
id: EXP03 name: SQL Injection mode: CONTAINER services: - vulnerable-web - database network: isolated: true
limits: memory: 512m cpu: 1
```

4. Runtime Modes

Mode	Use	Example
SIMULATION	No real vulnerable service required	RSA attack analysis, frequency analysis
CONTAINER	Vulnerable application/service	SQLi, XSS, authentication, path traversal
NETWORK_LAB	Multiple isolated nodes/services	Packet sniffing, ARP/SYN/ICMP demonstrations

5. Docker Environment

A Dockerfile defines the software environment for an individual image. For a lab that needs several components (for example Node.js + PostgreSQL), Docker Compose is generally cleaner: one service for the application and another for the database.

```
services: web: build: ./target ports: - "3000:3000" depends_on: - db db: image: postgres:... environment:
POSTGRES_DB: labdb POSTGRES_USER: lab POSTGRES_PASSWORD: lab
```

Expose only the ports required by the lab. Keep the lab network isolated from unrelated systems.

6. Electron Lab Manager

Electron should orchestrate the environment rather than contain every attack implementation.

```
Start Lab ↓ Validate lab manifest ↓ Build/pull required image ↓ Create isolated network ↓ Start services ↓
Show localhost target + terminal ↓ Collect logs/metrics ↓ Stop / Reset / Remove Lab
```

7. Embedded Terminal

For terminal-oriented experiments, the UI can contain a terminal emulator such as xterm.js, while the actual command execution occurs inside the lab environment. Avoid giving the lab terminal unrestricted host-shell access. Commands should execute inside the intended container or sandbox.

8. Attack Simulation Design Pattern

Every experiment should follow the same educational lifecycle:

```
Theory ↓ Lab setup ↓ Observe normal behavior ↓ Perform controlled attack/simulation ↓ Observe changed behavior ↓ Explain root cause ↓ Apply mitigation ↓ Repeat test ↓ Show result + activity log ↓ Quiz / assessment
```

9. Examples for the Assigned Experiments

Experiment	Local implementation concept
Packet Sniffing	Isolated lab network + packet capture/visualization
ARP Spoofing	Multiple isolated nodes + controlled ARP-state demonstration
SQL Injection	Vulnerable Node.js app + PostgreSQL + query visualization
Vulnerable Authentication	Local login service with intentionally weak authentication
Brute Force / Frequency Analysis	Local dataset/cipher and controlled computation
Path Traversal	Vulnerable file-serving application with a sandboxed file tree
Buffer Overflow	Purpose-built vulnerable native program inside a constrained container
DoS / DDoS	Controlled resource/traffic simulation against an isolated target
XSS	Vulnerable local web application + safe test payloads
Trojan Horse	Behavior/detection simulation rather than uncontrolled malware
ICMP Flood	Isolated network and bounded traffic demonstration
Malicious Program Detection	Signature + behavioral indicators on a controlled sample set
SYN Flood	Isolated target network with bounded packet simulation
RSA Known Ciphertext Attack	Local cryptographic computation and visualization

10. Safety and Isolation Requirements

• Run intentionally vulnerable services only inside the lab environment.

• Prefer an isolated Docker network for network-security experiments.

• Do not design the lab to attack arbitrary Internet hosts or the student's LAN.

• Put CPU, memory, process and execution-time limits on resource-intensive simulations.

• Provide Stop Lab and Reset Lab controls.

• Clean up containers, networks and temporary volumes after completion.

• Do not execute untrusted student-submitted code directly on the host OS.

11. Cleanup Lifecycle

```
Student clicks "Stop Lab" ↓ Stop services ↓ Remove lab containers ↓ Remove temporary network ↓ Remove temporary volumes (when appropriate) ↓ Return to clean state
```

12. Required UI

Screen/Component	Purpose
Experiment page	Aim, objectives, theory and procedure
Lab status	Stopped / Starting / Running / Error
Target panel	Open the local vulnerable application
Terminal	Execute permitted lab commands inside the sandbox
Metrics	CPU, RAM, requests/packets, logs as applicable
Activity log	Record experiment events and results
Reset/Stop	Safely terminate and clean the environment
Quiz	Assessment after the simulation

13. SQL Injection Reference Implementation

For the SQL Injection experiment, the college guideline specifically expects a simulated login/database environment, username/password/SQL input, authentication result, generated SQL query, database response, vulnerability status, attack simulation, vulnerability detection, prevention/parameterized-query demonstration, and activity log.

A suitable local architecture is: Electron UI → local Node.js vulnerable application → PostgreSQL container. The same lab can then demonstrate the vulnerable behavior and the parameterized-query mitigation without requiring a cloud database.

14. Testing Checklist

• Fresh installation: verify the lab starts without manual dependency installation.

• Normal case: verify the target works before the simulation.

• Attack case: verify the expected vulnerable behavior is reproduced.

• Mitigation case: verify the vulnerability is reduced/eliminated after the fix.

• Reset case: verify the lab returns to a clean state.

• Failure case: verify Docker/image/network errors are shown clearly.

• Isolation case: verify the experiment cannot reach unrelated host/network resources.

• Integration case: verify the module can be added without modifying other experiments.

15. Mapping to the College Guidelines

The uploaded assignment requires each virtual lab, as applicable, to contain aim/objectives, theory, step-by-step procedure, interactive simulation/implementation, assessment/quiz, references and feedback, and it emphasizes that the interface must be interactive rather than static. ■filecite■turn0file0■L19-L31■

The assignment also expects experiment modules to be self-contained and to include source code, experiment content, test cases, expected outputs, evaluation questions, quiz questions and an integration README. ■filecite■turn0file0■L284-L290■

Final Design Principle

Electron = Lab Controller/UI
Docker = Isolated Execution Environment
Docker Compose = Multi-service Lab
Terminal = User Interaction Layer
Website/Cloud = Distribution + Documentation + Optional Progress